Py-01 · Introduzione a Python

Indice

1 Cos'è Python **2**
- 1.1 Concetti di base . . . 2
- 1.2 Python: a cosa serve . . . 2
- 1.3 Perché Python . . . 3
- 1.4 Caratteristiche chiave . . . 3
- 1.5 Installare Python . . . 3
- 1.6 Codice sorgente . . . 4
- 1.7 Commenti . . . 4

2 Variabili **4**
- 2.1 Variabili - convenzioni . . . 6
- 2.2 Tipi numerici e operatori aritmetici . . . 6
- 2.3 Booleani . . . 7
- 2.4 Stringhe . . . 10

3 Input e output **11**
- 3.1 Primo esempio: print() e input() . . . 11
- 3.2 Le funzioni `int()` e `float()` . . . 13
- 3.3 Primi esempi di programma . . . 15

4 Tipi di dato **15**
- 4.1 La funzione type() . . . 15
- 4.2 Rappresentazione approssimata dei float . . . 16
- 4.3 Dove vengono memorizzati i dati? . . . 16
- 4.4 Interi piccoli: la cache di CPython . . . 16
- 4.5 Keyword . . . 18

5 Oggetti, metodi **19**
- 5.1 Confronto tra oggetti: valore e identità . . . 19

6 Istruzioni **21**
- 6.1 Cos'è un'istruzione . . . 21
- 6.2 Flusso di esecuzione sequenziale . . . 22

1 Cos'è Python

1.1 Concetti di base

- **Linguaggio di programmazione**
- **Codice sorgente**
- **Programma**
- **Interprete**
- **Compilatore**
- **Python**

Un **linguaggio di programmazione** è un linguaggio formale usato per scrivere istruzioni che un computer può eseguire. Consente di rappresentare in forma strutturata processi computazionali, permettendo la traduzione del codice in **operazioni comprensibili ed eseguibili da una macchina**.

Il **codice sorgente** è il testo di un programma scritto in un linguaggio di programmazione leggibile dagli esseri umani (come Python, C o Java). Contiene le istruzioni che descrivono il comportamento del programma e costituisce la forma originaria del software prima della sua esecuzione.

Un **programma** (o applicazione, spesso abbreviato in **app**) è un insieme organizzato di istruzioni scritte in un linguaggio di programmazione che, una volta eseguite da un computer, permettono di svolgere uno specifico compito o risolvere un determinato problema.

Un **interprete** è un programma che esegue il codice sorgente traducendolo ed eseguendolo *istruzione per istruzione* durante l'esecuzione. Esempio: Python.

Un **compilatore** è un programma che traduce *l'intero codice sorgente in anticipo* in un linguaggio macchina o in un codice intermedio, generando un file eseguibile o oggetto. L'esecuzione avviene successivamente, senza bisogno del codice sorgente originale. Esempio: C.

1.2 Python: a cosa serve

Python è un **linguaggio di programmazione** di alto livello, *interpretato*, *interattivo* e *orientato agli oggetti*.

Con Python si possono realizzare piccoli programmi e script di base ma anche programmi molto complessi.

Esempi

- Creare applicazioni desktop, incluse applicazioni GUI e strumenti da riga di comando (CLI)
- Eseguire analisi matematiche e scientifiche dei dati, e rappresentazione grafica dei risultati
- Sviluppare applicazioni web
- Amministrare sistemi informatici e automatizzare attività
- Svolgere attività DevOps

1.3 Perché Python

- **Sintassi leggibile e adatta ai principianti**
- **"Batteries included":** la libreria standard include moduli per gestione dei file, networking, matematica, data e ora, test e molto altro.

- **Grande comunità e molte risorse:** esiste un vasto ecosistema di tutorial, video, forum e meetup locali per ogni argomento e livello di competenza.

- **Ampia diffusione nel mondo reale**
- **Versatile e scalabile**
- **Gratuito e multipiattaforma**

- **Open source:** il codice sorgente di Python è rilasciato con la Python Software Foundation License Version 2, che garantisce ampi diritti di utilizzo, modifica e distribuzione, anche in software proprietari. Chiunque può contribuire al suo sviluppo.

1.4 Caratteristiche chiave

- **Interpretato**
- **Multiparadigma:** permette di scrivere codice in diversi stili, tra cui orientato agli oggetti, imperativo e funzionale.

- **Tipizzazione dinamica:** il tipo è associato ai valori, non alle variabili, e viene controllato durante l'esecuzione; non serve dichiararlo.

- **Fortemente tipizzato:** non consente operazioni non sicure tra tipi incompatibili senza segnalarle.

1.5 Installare Python

Esistono diversi modi per installare Python e vari strumenti per scrivere il codice sorgente. La scelta dipende dalle proprie preferenze, dal sistema operativo utilizzato e dagli obiettivi di sviluppo.



THONNY

Uno strumento gratuito, multipiattaforma e semplicissimo da installare è https://thonny.org/.

Non è adatto a gestire progetti complessi ma **ottimo per le esigenze di un corso introduttivo**.

Alcuni esempi di ambienti di sviluppo (IDE) e distribuzioni:

- **PyCharm** (www.jetbrains.com/pycharm) – ambiente di sviluppo completo e professionale, adatto sia ai principianti sia a progetti complessi
- **Anaconda** (www.anaconda.com) – distribuzione molto diffusa per l'analisi dei dati, che include strumenti come Spyder e JupyterLab
- **Wing IDE** (www.wingware.com) – ambiente di sviluppo leggero e professionale dedicato a Python

Per i dettagli sull'installazione e la configurazione, si consiglia di consultare la documentazione ufficiale dei programmi scelti, relativa al proprio sistema operativo.

Strumenti online, utilizzabili subito **senza necessità di installazione**

- GOOGLE COLAB (https://colab.research.google.com/) **immediatamente pronto all'uso**, è sufficiente un account Google.
- Altri servizi online (cercate python online)

1.6 Codice sorgente

Il codice sorgente in Python è un semplice file di testo con estensione `.py`.

Un programma Python può essere costituito da uno o più file che contengono il codice sorgente. Oltre a questi, possono essere presenti altri file che archiviano informazioni aggiuntive, come immagini, icone e altre risorse utili al funzionamento del programma.

1.7 Commenti

I commenti sono porzioni di testo presenti all'interno del codice ma ignorate dall'interprete Python durante l'esecuzione.

Vengono utilizzati per descrivere il funzionamento del codice, facilitando una rapida comprensione da parte degli sviluppatori, oltre a chiarire le motivazioni che hanno portato alla sua scrittura in un determinato modo.

```
# Questo è un commento!
```

2 Variabili

In Python, le variabili sono **nomi associati a un determinato oggetto**.

Le variabili contengono un *riferimento*, o puntatore, all'indirizzo di memoria in cui l'oggetto è memorizzato. Una volta che a una variabile viene assegnato un oggetto, è possibile accedere a quest'ultimo utilizzando il nome della variabile.

> **Assegnazione**
>
> L'**assegnazione** in Python è l'operazione che associa un valore a una variabile.

Si effettua utilizzando l'operatore `=` (uguale), che *non indica uguaglianza matematica*, ma un'associazione di riferimento tra un nome (la variabile) e un oggetto in memoria.

```
x = 10
nome = "Alice"
pi_greco = 3.14
```

In questi esempi:

- la variabile `x` fa riferimento al valore intero 10
- la variabile `nome` fa riferimento alla stringa "Alice"

- la variabile `pi_greco` fa riferimento al numero `3.14`

Cosa significa questa istruzione?

```
x = x + 1
```

Si calcola prima il lato destro (il valore attuale di `x` più 1), poi si associa il risultato al nome `x`. Non è un'equazione: è un aggiornamento.

! Assegnazione composta

Aggiornare una variabile partendo dal suo valore attuale è così frequente che Python offre una forma abbreviata:

```
x += 1      # equivale a x = x + 1
x -= 2      # equivale a x = x - 2
x *= 3      # equivale a x = x * 3
x /= 4      # equivale a x = x / 4
```

Gli operatori composti si scrivono senza spazi tra i due simboli: `+=`, non `+ =`.

La variabile contiene un riferimento

In Python, diversamente da altri linguaggi di programmazione, la variabile non 'contiene' direttamente il dato assegnato, ma memorizza un riferimento alla posizione in memoria in cui il dato è effettivamente memorizzato. Questa distinzione è sottile ma rilevante, la approfondiremo più avanti. Tuttavia, per semplicità, si usa comunemente l'espressione "**la variabile contiene il valore x**", pur sapendo che, dal punto di vista tecnico, l'affermazione non è corretta.

Il nome di una variabile dovrebbe offrire un'indicazione sul significato del valore assegnato.

Es.: `lista_utenti`, `numero_giorni`, `importo_totale`

I nomi delle variabili possono avere qualsiasi lunghezza e possono contenere lettere maiuscole e minuscole (`A-Z`, `a-z`), cifre (`0-9`) e il carattere di underscore (`_`). Python accetta anche lettere accentate (es. `età`), ma è preferibile evitarle.

Il primo carattere **non può essere una cifra**, deve essere necessariamente un carattere alfabetico o il carattere di underscore. Per ora però non utilizzeremo variabili che cominciano con il carattere di underscore.

Il nome delle variabili non può contenere spazi o altri caratteri speciali come `!£$#%`

! Maiuscole e minuscole contano

Python distingue maiuscole e minuscole (*case sensitive*): `nome` e `Nome` sono due variabili diverse, e `true` non è il valore booleano `True`.

```
nome = 'Ada'
print(Nome)     # NameError: name 'Nome' is not defined
x = true        # NameError: name 'true' is not defined
```

```
# ------------------------------------
# nomi leciti
# ------------------------------------
a
a122
lista_utenti

un_elemento
_a1  # per ora non useremo variabili che cominciano con _
__a2 # per ora non useremo variabili che cominciano con __

unElemento #lecita ma non conforme a PEP8
ABC #lecita, ma secondo PEP8 il maiuscolo è riservato alle COSTANTI

# ------------------------------------
# nomi non leciti
# ------------------------------------

3aa
valore!
somma totale
```

2.1 Variabili - convenzioni

Oltre alle regole da seguire obbligatoriamente (es. i nomi delle variabili non possono cominciare con un numero) esistono anche delle convenzioni che è bene adottare: nomi in minuscolo con parole separate da underscore (`snake_case`), es. `importo_totale`; nomi tutti maiuscoli per le costanti, es. `MAX_VALORE`.

https://peps.python.org/pep-0008/#function-and-variable-names

https://peps.python.org/pep-0008/

2.2 Tipi numerici e operatori aritmetici

Python fornisce supporto per numeri interi (**int**), numeri in virgola mobile (**float**) e numeri complessi (**complex**).

https://docs.python.org/it/3/library/stdtypes.html#numeric-types-int-float-complex

Gli operatori rappresentano operazioni, come addizione, sottrazione, moltiplicazione, divisione e così via.

Operatori aritmetici

```
# le 4 operazioni: addizione, sottrazione, moltiplicazione, divisione
5 + 3
5 - 3
5 * 3
5 / 3
```

```
# Divisione intera (floor division): arrotonda verso il basso
5 // 3

# Modulo (rende il resto della divisione)
5 % 3

# Elevamento a potenza
5 ** 3
```

! La divisione `/` restituisce sempre un float

```
6 / 2      # 3.0 (non 3)
7 / 2      # 3.5
6 // 2     # 3   (floor division: intero se gli operandi sono interi)
```

Per questo, nell'esempio con `input()` che vedremo più avanti, la metà di 12 viene stampata come `6.0`.
Se in un'operazione compaiono un `int` e un `float`, il risultato è un `float`:

```
2 * 1.5    # 3.0
3 + 2.0    # 5.0
```

! Precedenza degli operatori e parentesi

Come in matematica, le operazioni non vengono eseguite da sinistra a destra, ma secondo una **precedenza**:

1. `**` (potenza)
2. `*`, `/`, `//`, `%`
3. `+`, `-`

A parità di precedenza si procede da sinistra a destra. Le **parentesi** permettono di cambiare l'ordine.

```
2 + 3 * 4            # 14
(2 + 3) * 4          # 20

(-b + d) / (2*a)     # corretto
-b + d / 2*a         # ERRATO: vale -b + (d / 2) * a
```

Attenzione: `-3**2` vale `-9`, perché la potenza viene calcolata prima del segno meno. Scrivere `(-3)**2` per ottenere `9`.

2.3 Booleani

In Python i valori booleani sono solo due, `True` e `False`, e il loro tipo, `bool`, è una sottoclasse di `int`.

Per questo `True` si comporta come 1 e `False` come 0:

```
print(True + True)    # 2
print(True == 1)      # True
```

```
2
True
```

Usiamo i booleani per esprimere il **valore di verità** di un'espressione

```
a = True
print(a)
a = False
print(a)
```

```
True
False
```

Assegnazione e confronto

L'operatore `=` (uguale) indica ASSEGNAZIONE, l'operatore `==` (doppio uguale) è una richiesta di confronto, il cui risultato può essere `True` o `False`

Un **confronto** produce come risultato un valore booleano, che possiamo stampare o memorizzare in una variabile.

Esempio

```
x = 10
y = 20
answer = x == y
```

L'operazione `x == y` produce il risultato `False`, che viene assegnato alla variabile `answer`.

```
a = 2 < 5
print(a)
a = 4 > 10
print(a)
a = 4 <= 3
print(a)
a = 3 >= 3
print(a)
a = 5 == 4+2 # attenzione a confronto vs assegnazione
print(a)
a = 5 == 4+1
print(a)
a = 6 != 9
print(a)
a = (5 == 4) or (5 == 4+1) # or: vedi più avanti
print(a)
```

```
True
False
False
True
False
True
True
True
```

https://docs.python.org/it/3/library/stdtypes.html#boolean-type-bool

! Operatori di confronto

Un confronto tra due valori produce sempre un risultato booleano: `True` o `False`.

Operatore	Significato	Esempio	Risultato
`<`	minore	`2 < 5`	`True`
`>`	maggiore	`4 > 10`	`False`
`<=`	minore o uguale	`4 <= 3`	`False`
`>=`	maggiore o uguale	`3 >= 3`	`True`
`==`	uguale	`5 == 4`	`False`
`!=`	diverso	`6 != 9`	`True`

- `=` assegna un valore a una variabile, `==` confronta due valori: non confonderli.
- Gli operatori composti si scrivono senza spazi e nell'ordine indicato: `<=` e `>=`, non `=<` o `=>`.

! Operatori logici

Gli operatori logici combinano valori booleani (ad esempio i risultati di più confronti) e producono a loro volta `True` o `False`.

- `a and b` è `True` solo se **entrambi** sono `True`
- `a or b` è `True` se **almeno uno** dei due è `True`
- `not a` **inverte** il valore: `True` diventa `False` e viceversa

a	b	a and b	a or b	not a
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

- Esempio: `eta >= 18 and eta < 65` è `True` se l'età è compresa tra 18 e 64.
- Precedenza: prima `not`, poi `and`, infine `or`. Nel dubbio, usare le parentesi: `a or (b and c)`.
- Valutazione *short-circuit*: se il risultato è già determinato dal primo operando, il secondo non viene valutato (`False and ...` è sempre `False`, `True or ...` è sempre `True`).

2.4 Stringhe

Le stringhe sono sequenze di caratteri definite usando virgolette (singole, doppie o triple).

https://docs.python.org/it/3/library/stdtypes.html#text-sequence-type-str

! Apostrofi e virgolette nelle stringhe

Se la stringa contiene un apostrofo, va racchiusa tra virgolette **doppie**; se contiene virgolette doppie, tra virgolette **singole**.

```
s = 'l'albero'        # ERRORE: la stringa finisce dopo 'l'
# SyntaxError: unterminated string literal

s = "l'albero"        # corretto
t = 'disse "ciao"'    # corretto
```

L'operatore più (+) concatena le stringhe.

```
a = 'hello'
b = "world!"

c = a+b
print(c)
```

```
helloworld!
```

La funzione `len()` applicata ad una stringa rende la lunghezza della stringa, cioè il **numero di caratteri**. Diversamente dal C, in Python non esiste un carattere nascosto di fine stringa. Python conta solo i caratteri che vediamo davvero. Gli spazi contano!

```
s = 'hello world'
print(len(s))
```

```
11
```

Per ora possiamo pensare a `len()` come a un *comando* di Python che conta quanti caratteri ci sono in una stringa. Più avanti vedremo meglio cosa significa dire che `len()` è una 'funzione'.

Un altro 'comando' applicabile alle stringhe è `upper()`, che permette di ottenere una nuova stringa in maiuscolo. `upper()` si utilizza in questo modo:

```
s1 = 'Hello World!'
s2 = s1.upper()

print(s1)

print(s2)
```

```
Hello World!
HELLO WORLD!
```

Si noti che

- `upper()` si scrive *dopo* la stringa, separato da un punto (*dot notation*). Un comando usato in questo modo si chiama **metodo**; vedremo più avanti cosa significa esattamente.
- la stringa originale `s1` non viene modificata.
- `s2` contiene la versione 'maiuscola' di `s1`.

- solo le lettere minuscole vengono trasformate: spazi, numeri, simboli e lettere già maiuscole restano invariati.

Python fornisce anche altri tipi di dati. Li vedremo in seguito, e approfondiremo le caratteristiche dei tipi di dato fin qui illustrati.

3 Input e output

3.1 Primo esempio: print() e input()

Python mette a disposizione delle istruzioni che permettono di inviare informazioni in uscita (**output**) e di acquisire informazioni dall'esterno (**input**). I canali standard di input e output sono la tastiera e lo schermo, ma è possibile utilizzare tutte le periferiche I/O a disposizione.

```
print('ciao, inserisci qualcosa da tastiera')
# scrive sullo schermo

a = input()
# acquisisce da tastiera e inserisce il valore nella variabile

print('Ciao, hai scritto', a)
# scrive sullo schermo
```

La sequenza di caratteri `'Ciao, hai scritto'` è una stringa.

`print('Ciao, hai scritto')` visualizza sullo schermo l'esatta sequenza di caratteri.

`a` è una variabile.

Notate che all'interno di `print()` la variabile `a` è scritta *senza* gli apici che delimitano una stringa. Questo è importante: significa che `print()` mostrerà il valore memorizzato nella variabile `a` e non semplicemente la lettera `a`. Se invece avessimo scritto `print('a')`, il programma avrebbe stampato il carattere `a` anziché il valore della variabile.

! `input()` con messaggio

Il messaggio da mostrare all'utente si può passare direttamente a `input()`, senza una `print()` separata:

```
nome = input('Come ti chiami? ')
```

equivale a

```
print('Come ti chiami?')
nome = input()
```

con una differenza: il cursore resta sulla stessa riga del messaggio (per questo conviene lasciare uno spazio finale nel messaggio).
Si può combinare con la conversione: `eta = int(input('Quanti anni hai? '))`

Ancora su `print()`

- Gli argomenti di `print()` sono una **sequenza di elementi separati da virgola**.
- Ogni elemento può essere un **valore** o qualcosa che **produce un valore**.
  - una **stringa**
  - un **numero**
  - una **variabile**
  - un'**espressione** (per ora ci riferiamo alle espressioni matematiche)
- Le **stringhe** vengono stampate così come sono.
- Le **espressioni matematiche** vengono prima calcolate, poi stampate.
- Le **variabili** vengono sostituite con il loro valore.
- Tra gli elementi separati da virgola, `print()` inserisce automaticamente uno **spazio**.
- Alla fine della stampa, `print()` va automaticamente **a capo**.

`print()` offre molte altre opzioni per controllare l'output: formattazione del testo, dei numeri e modalità di stampa.

Esempio

```
a = 10.1
b = 2
s = "Buon giorno!"

print(s,"Il risultato è", a*b)
```

```
Buon giorno! Il risultato è 20.2
```

Riferimento ufficiale: Python Tutorial – Input and Output

https://docs.python.org/3/tutorial/inputoutput.html

Non tutte le operazioni sono possibili su tutti i tipi di dati!

```
print('ciao, inserisci un numero!')
a = input() # acquisisce da tastiera
b = a/2  # ERRORE! a è una stringa

print('hai inserito', a)
print('la metà è', b)
```

`input()` restituisce sempre una **stringa**, anche se l'utente digita un numero. Occorre convertire la stringa in numero.

È molto importante capire i messaggi d'errore dell'interprete. Ci guidano verso la correzione dell'errore (anche all'esame!).

Altro esempio: cosa succede?

```
print('ciao, inserisci un numero!')
a = input()

print(a+a)
```

3.2 Le funzioni `int()` e `float()`

Le funzioni `int()` e `float()` permettono di convertire una stringa in intero o float (se la conversione è possibile).

`float` significa 'virgola mobile'.

Un numero in virgola mobile viene rappresentato mediante **mantissa** ed **esponente**, come in $3.14 = 314 \cdot 10^{-2}$.

È il modo in cui il calcolatore memorizza (e approssima) i numeri reali. Nel calcolatore le potenze sono espresse in base 2 e non in base 10.

I numeri reali possono essere rappresentati solo in maniera approssimata (perché?).

Conseguenza della rappresentazione interna approssimata:

```
print(1-(1/3+1/3+1/3))

print(1-1/3-1/3-1/3)
```

```
0.0
1.1102230246251565e-16
```

Le due espressioni sono matematicamente equivalenti, ma i risultati sono diversi: con i float l'ordine delle operazioni influisce sugli arrotondamenti.

! Arrotondare con `round()`

A causa dell'approssimazione, alcuni calcoli producono risultati "strani", ad esempio con gli importi in euro:

```
prezzo = 1.4 * 3
print(prezzo)              # 4.199999999999999
print(round(prezzo, 2))    # 4.2  (arrotonda a 2 cifre decimali)
print(round(3.7))          # 4    (senza il secondo argomento:
 ↪  intero)
```

`round()` serve per **mostrare** il risultato in modo leggibile; il calcolo interno resta approssimato.

Attenzione: i valori che terminano esattamente in `.5` vengono arrotondati al numero **pari** più vicino: `round(2.5)` vale 2, `round(3.5)` vale 4.

Esempio di input con conversione:

```
print('ciao, inserisci un numero!')

a = input() # acquisisce da tastiera
a = int(a) # oppure float(a) - converte in numero

b = a/2

print('hai inserito', a)
print('la metà è', b)

# alternativa: input e converto inline
# a = int(input())
```

Non sempre le conversioni sono possibili...

```
a = "12"
b = "13.55"
c = "ciao"

a1 = int(a) # 12 rappresentato come intero
a2 = float(a) # 12.0 rappresentato come float

b1 = int(b) # ERRORE. Tentativo di conversione a intero
b2 = float(b) # 13.55 rappresentato come float

c1 = int(c) # ERRORE. Tentativo di conversione a intero
c2 = float(c) # ERRORE. Tentativo di conversione a float
```

! Conversione inversa: la funzione `str()`

`str()` converte un numero in stringa. Serve, ad esempio, per concatenare un numero con del testo usando `+`:

```
eta = 20
print('Hai ' + eta + ' anni')        # ERRORE
# TypeError: can only concatenate str (not "int") to str

print('Hai ' + str(eta) + ' anni')   # Hai 20 anni
print('Hai', eta, 'anni')            # Hai 20 anni
```

3.3 Primi esempi di programma

```
# il mio primo programma
print("come ti chiami?")
nome_utente = input()
stringa_finale = 'CIAO! ' + nome_utente

print(stringa_finale)
```

```
# Secondo programma
print('dammi primo numero')
v1_s = input() # stringa
print('dammi secondo numero')
v2_s = input()

v1 = float(v1_s) # converto la stringa in float
v2 = float(v2_s)

risultato = v1 + v2

print('ecco la somma!')
print(risultato)
```

4 Tipi di dato

Python gestisce diversi tipi di dati (intero, float, stringa, ecc), e ogni tipo viene archiviato e manipolato in modo specifico.

Python è un linguaggio **dinamicamente tipizzato**, il che significa che non è necessario dichiarare esplicitamente il tipo di una variabile: Python lo deduce automaticamente in base al valore assegnato.

Tuttavia, il tipo di dato è fondamentale perché determina:

- come i dati vengono memorizzati in memoria
- quali operazioni possono essere eseguite su di essi
- come Python li gestisce internamente

4.1 La funzione type()

La funzione `type()` restituisce il tipo (la classe) dell'oggetto passato come argomento.

```
a = 10
b = 11.11
c = 'hello'
```

```
print(a, ' tipo di dato: ', type(a))
print(b, ' tipo di dato: ', type(b))
print(c, ' tipo di dato: ', type(c))


# 10  tipo di dato:  <class 'int'>
# 11.11  tipo di dato:  <class 'float'>
# hello  tipo di dato:  <class 'str'>
```

4.2 Rappresentazione approssimata dei float

Conseguenza pratica: i numeri reali possono avere infinite cifre decimali; la rappresentazione mediante `float` è necessariamente approssimata

```
a = 1-1/3-1/3 -1/3
print(a)  # teoricamente 0
print(1/a) # teoricamente impossibile
```

4.3 Dove vengono memorizzati i dati?

Come detto, la variabile contiene un riferimento all'area di memoria in cui è memorizzato il dato. Per ottenere l'indirizzo di memoria a cui una variabile fa riferimento possiamo usare la funzione `id()`

```
a = 10
# memorizza 10 in memoria. `a` è un riferimento a quell'area di memoria.

b = a
# crea un nuovo riferimento alla stessa area di memoria.

print(hex(id(a)), hex(id(b)))
```

Nell'esempio utilizziamo anche la funzione `hex()` per trasformare l'indirizzo in base esadecimale (base 16). L'esempio mostra che le due variabili fanno riferimento allo stesso indirizzo.

In realtà la funzione `id()` restituisce un valore univoco per ogni oggetto presente nel programma https://docs.python.org/it/3/library/functions.html#id. Python ha varie implementazioni; nella versione più diffusa (implementazione `CPython`), `id()` restituisce un valore che, al momento in cui vengono scritte queste note, corrisponde all'indirizzo di memoria dell'oggetto. Ai fini di questa discussione non è fondamentale se il valore restituito da `id()` sia il vero indirizzo di memoria. L'importante è che `id()` fornisca un riferimento univoco per ogni oggetto.

4.4 Interi piccoli: la cache di CPython

In CPython i numeri interi da -5 a 256 vengono creati una sola volta e riutilizzati, attraverso un meccanismo di *caching* (spesso chiamato informalmente `interning`). Ogni volta che si assegna

uno di questi valori a una variabile, CPython riutilizza la stessa istanza in memoria, quindi tutti i riferimenti a questi numeri puntano allo stesso oggetto.

https://docs.python.org/3/c-api/long.html#c.PyLong_FromLong

Attenzione: questo comportamento non è una caratteristica del linguaggio Python, ma un dettaglio implementativo dell'interprete. Se la versione di Python in uso è basata su CPython, in genere gli interi in un certo intervallo vengono memorizzati attraverso il meccanismo di `interning`; tuttavia questo comportamento non è garantito dal linguaggio e può variare tra implementazioni o versioni diverse.

```
a = 10
# memorizza 10 in memoria. `a` è un riferimento a quell'area di memoria.

b = int("10")
# Python riusa la stessa area di memoria

print(hex(id(a)), hex(id(b)))
```

```
a = 123456789123456789
# memorizza il valore in memoria. `a` è un riferimento a quell'area di
 ↪  memoria.

b = a + 1
c = b - 1
# a e c hanno lo stesso valore,
# ma Python NON riusa la stessa area di memoria
# (non sono interi piccoli)
print(a == c)
print(hex(id(a)), hex(id(c)))
```

```
True
0x1059fde70 0x1059fe290
```

```
a = 3.1
b = 3.1
# per i float non viene utilizzato il meccanismo di interning

print(hex(id(a)), hex(id(b)))
```

```
0x105ad01b0 0x1059fe810
```

Il risultato dipende dall'implementazione. Provate `a = float("3.1")` e `b = float("3.1")`.

Questo esempio mostra concretamente che la variabile **non** contiene direttamente il valore, ma solo un riferimento. Se cambio il valore, cambio il riferimento. Questo meccanismo è diverso dal meccanismo adottato dal linguaggio `C`

Esempio in Python

```
a = 1
print(hex(id(a)))
```

```
# memorizza 1 in memoria. `a` è un riferimento a quell'area di memoria.

a = a +1
print(hex(id(a)))
# memorizza  2 in una nuova area di memoria.
# `a` è un riferimento alla nuova area di memoria.
```

```
0x102e1afa8
0x102e1afc8
```

Esempio in C

```
    int a;
    a = 10;
    printf("%p\n",&a ); //0x16b446f6c
    a = a + 1;
    printf("%p\n",&a ); //0x16b446f6c
```

4.5 Keyword

Le parole chiave (**keyword**) sono termini riservati dal linguaggio che hanno un significato specifico e non possono essere usati come nomi (identificatori) di variabili o funzioni. Python ha un insieme di parole chiave fissato per ciascuna versione del linguaggio.

Per ottenere l'elenco completo delle parole chiave:

```
>>> help("keywords")

Here is a list of the Python keywords.  Enter any keyword to get more help.

False               class               from                or
None                continue            global              pass
True                def                 if                  raise
and                 del                 import              return
as                  elif                in                  try
assert              else                is                  while
async               except              lambda              with
await               finally             nonlocal            yield
break               for                 not
```

Ognuna di queste parole chiave gioca un ruolo nella sintassi di Python. Sono **parole riservate** che hanno significati e scopi specifici nel linguaggio. **Non usatele come nomi di variabili nel codice!**

Errore subdolo

L'istruzione

```
print = 10
```

utilizzerà il nome della funzione `print` come **nome di una variabile**, impedendo l'uso della funzione `print()` fino al riavvio dell'interprete (lo stesso vale per gli altri nomi predefiniti, come `input`, `len`, `type`). Le keyword vere invece non possono essere usate come nomi: `if = 10` produce un errore di sintassi.

5 Oggetti, metodi

In Python tutti i valori che assegniamo a una variabile sono **oggetti**. Un oggetto può essere visto come una combinazione di **dato + comportamento**: da una parte contiene un valore (per esempio una stringa o un numero), dall'altra mette a disposizione alcune **operazioni che possono essere applicate a quel valore**. Queste operazioni si chiamano **metodi** e si richiamano usando la **dot notation**. Ad esempio, se `s` è una stringa, con `s.upper()` chiediamo all'oggetto stringa di restituire una versione tutta in maiuscolo.

Un oggetto non è solo un valore memorizzato in una variabile, ma anche **un insieme di funzionalità che sanno operare su quel valore**.

Ricordiamo inoltre che, come visto in precedenza, in Python **la variabile non contiene direttamente il dato** (come avviene in linguaggi come C), ma **contiene un riferimento all'area di memoria in cui l'oggetto è memorizzato**. Per semplicità nel linguaggio comune diremo spesso che *"la variabile contiene l'oggetto"*, ma è bene essere consapevoli che si tratta di **un'approssimazione**.

5.1 Confronto tra oggetti: valore e identità

In Python esistono due modi diversi per confrontare valori: `==` e `is`.

L'operatore `==` verifica se **due oggetti hanno lo stesso valore**. Ad esempio, due stringhe con gli stessi caratteri risultano uguali anche se sono oggetti distinti (e quindi occupano zone diverse della memoria).

L'operatore `is`, invece, verifica se **due variabili fanno riferimento allo stesso oggetto in memoria**. In altre parole controlla l'identità dell'oggetto, non il suo contenuto.

Nella maggior parte dei casi, quando vogliamo confrontare dei valori, possiamo usare `==`. L'operatore `is` viene usato più raramente, quando vogliamo verificare se due variabili puntano **esattamente allo stesso oggetto**.

Ecco alcuni esempi che mostrano la differenza tra `==` e `is`. Riprenderemo questi esempi quando introdurremo altri tipi di dato.

```
a = 10
b = a

print(a == b)   # True
print(a is b)   # True
```

```
True
True
```

Qui entrambi i confronti risultano `True`. `a` e `b` si riferiscono allo stesso oggetto.

```python
a = int("30")
b = int("30")

print(a == b)   # True
print(a is b)   # probabilmente True
```

```
True
True
```

Gli interi sono creati separatamente, ma se la versione di Python in uso implementa il meccanismo di `interning` i valori interi 'piccoli' possono essere rappresentati dallo stesso oggetto in memoria. In tal caso anche `is` rende `True`.

```python
a = int("1000")
b = int("1000")

print(a == b)   # True
print(a is b)   # False
```

```
True
False
```

I valori sono uguali (`==`), ma si tratta di **due oggetti distinti**, quindi `is` è `False`.

```python
x = float("3.14")
y = float("3.14")

print(x == y)   # True
print(x is y)   # False
```

Anche qui i valori sono uguali, ma gli oggetti sono diversi. Per i `float` il meccanismo di `interning` non viene utilizzato.

```python
s1 = "ciao"
s2 = "ciao"

print(s1 == s2)   # True
print(s1 is s2)   # spesso True
```

```
True
True
```

Le stringhe possono essere **riutilizzate automaticamente** da Python, quindi a volte `is` restituisce `True`. Tuttavia questo comportamento **non è garantito**.

```python
s1 = "ciao"
s2 = "ci"
s2 = s2+"ao"

print(s1 == s2)   # True
```

```
print(s1 is s2)   # False
```

```
True
False
```

Le due stringhe hanno lo **stesso contenuto**, quindi `==` restituisce `True`. Tuttavia `s2` viene costruita **a runtime** tramite concatenazione, quindi è **un oggetto diverso** da `s1`. Per questo `s1 is s2` restituisce `False`.

5.1.1 L'oggetto `None`

In Python esiste un oggetto speciale chiamato **`None`**. Viene spesso utilizzato per indicare che **una variabile non ha ancora un valore significativo** oppure che **una funzione non restituisce alcun risultato**. `None` è un oggetto unico (un *singleton*): in tutto il programma esiste **una sola istanza di `None`**. Per questo motivo, quando vogliamo verificare se una variabile vale `None`, è buona pratica usare l'operatore `is`.

```
x = None

print(x is None)  # True

x = 1 + 2
print(x is None)  # False
```

```
True
False
```

6 Istruzioni

6.1 Cos'è un'istruzione

Un'istruzione (statement) è un'unità eseguibile di codice che Python può interpretare ed eseguire. Le istruzioni possono includere parole chiave, operatori e funzioni.

Alcuni esempi semplici di istruzioni sono:

- l'assegnazione di un valore a una variabile,
- la stampa di un risultato sullo schermo

Esempio:

```
a = 10
b = input()
c = 2 * a
print('Risultato', c)
```

6.2 Flusso di esecuzione sequenziale

Quando Python esegue un programma, interpreta ed esegue le istruzioni in modo **sequenziale**, cioè dall'alto verso il basso, seguendo esattamente l'ordine in cui sono state scritte nel codice.

Questo comportamento è chiamato **flusso di esecuzione sequenziale**. L'ordine delle istruzioni è quindi fondamentale, perché determina il comportamento del programma e i risultati prodotti.

Successivamente vedremo che è possibile **alterare il flusso sequenziale delle istruzioni** utilizzando particolari costrutti del linguaggio, come condizioni, cicli e funzioni.

Questi strumenti permettono di eseguire alcune istruzioni solo in determinate situazioni, di ripeterle più volte o di organizzarle in blocchi riutilizzabili.

ESEMPI ED ESERCIZI

1) **`costo_colazione_1`**

Scrivere un programma che chieda all'utente il numero di paste mangiate ed il costo unitario. Il programma deve calcolare e stampare il costo finale. Stampare anche il tipo di dato e l'indirizzo di memoria

```
print('quante paste hai mangiato?')
stringa_numero_paste = input()
numero_paste = int(stringa_numero_paste)
print('costo unitario?')
stringa_costo_unitario = input()
costo_unitario = float(stringa_costo_unitario)

print('tipo della var stringa_numero_paste:', type(stringa_numero_paste))
print('tipo della var numero_paste:', type(numero_paste))
print('tipo della var costo_unitario:', type(costo_unitario))
print('locazione di memoria della var costo_unitario:',
 ↪  hex(id(costo_unitario)))


costo_totale = numero_paste * costo_unitario
print('costo totale', costo_totale)
```

2) **`hello_1`**

Scrivere un programma che chieda all'utente il suo nome, il suo cognome e lo saluti stampando la stringa 'Ciao' seguita da nome e cognome.

Variante: stampare il numero di caratteri totali utilizzato per nome e cognome. Suggerimento: `len(s)` rende la lunghezza di una stringa.

3) **`rettangolo_1`**

Scrivere un programma che chieda all'utente la lunghezza dei lati di un rettangolo e stampi l'area e il perimetro.

4) **`triangolo_1`**

Scrivere un programma che chieda all'utente base e altezza di un triangolo e stampi l'area.

5) **`triangolo_2`**

Scrivere un programma che chieda all'utente l'area di un triangolo rettangolo isoscele e stampi il perimetro.

Suggerimento:

```
import math
math.sqrt(x)  # radice quadrata di x
```

permette di calcolare la radice quadrata.

6) **`triangolo_3`**

Scrivere un programma che chieda all'utente la lunghezza dei lati di un **triangolo** e stampi l'area e il perimetro

7) **`eq_1`**

- Scrivere un programma che chieda all'utente i coefficienti di un'equazione di secondo grado nella forma $a \cdot x^2 + b \cdot x + c = 0$ e trovi le radici. Si assuma che l'equazione ammetta radici reali e distinte.

8) **`conversione_4bit_1`**

- Scrivere un programma che chieda all'utente il valore delle 4 cifre di un numero intero espresso in base 2 e renda il valore in base 10. Il programma calcoli anche il prodotto per 2 e la divisione (intera) per 2 di tale numero, senza utilizzare direttamente le operazioni di prodotto e divisione, ma utilizzando la proprietà di shift a destra e sinistra.